Privacy Enhanced Knowledge Graph

MTP Project Phase 1

Jatothu Srinivas Nayak – CS24M125

Supervisors : Dr. Sridhar Chimalakonda & Dr. G Ramakrishna

Computer Science & Engineering

Table of Contents

1. Background, Motivation, and Literature Review
2. Problem Statement & Objectives
3. Methodology Overview
4. Phase-1 Implementation
5. Results & Evaluation
6. Privacy Leakage Analysis
7. Contributions & Limitations
8. Future Work
9. Proposed PEKG Architecture(Phase-2)
10. Sanitization Strategy (PEKG)(Phase-2)
11. Conclusion

Background: Code Understanding Challenges

- Modern software systems are large, interconnected, and evolve rapidly.
- Developers spend significant time understanding unfamiliar code before modifying it.
- Distributed behavior across files, modules, APIs, and services increases cognitive load.
- Real-world codebases contain mixed languages, third-party dependencies, and auto-generated code.

Background: Why Knowledge Graphs?

- Knowledge Graphs (KGs) model software as nodes (files, functions, classes) and edges (calls, imports, dependencies).
- Provide a structured, navigable representation of large codebases.
- Enable reasoning across files and modules important for summarization, impact analysis, and comprehension.

Background: LLMs and Privacy Issues

- LLMs (GPT, CodeT5, StarCoder) generate high-quality summaries but require raw source code.
- Raw source code contains:
  - Sensitive identifiers (service names, API endpoints)
  - Internal architecture patterns
  - PII in commit metadata
  - Tokens, URLs, and configuration secrets
- Uploading internal code to external LLMs poses major privacy and IP risks.
- Even providing a raw KG (without source code) still leaks identifiers and architectural cues.

Related Work: Structural Anonymization

Bellomarini et al. (2024) (Bellomarini et al., 2024)

- **Problem:** Publishing enterprise KGs leaks sensitive structure; simple anonymization fails as attackers can infer identities through graph shape.
- **Implementation:** Proposed (k, x)-isomorphism with **KLONE** and **KGUARD** to generate synthetic subgraph clones while preserving semantics.
- **Limitations:**
  - Not designed for software-engineering graphs or AST-derived structures.
  - Cannot handle code-level relationships such as CALLS, IMPORTS, or dependency chains.
- **Motivation for PEKG:** Shows the need for semantic retention while masking sensitive identifiers in program KGs.

Related Work: Differentially Private KG Embeddings

Zhang et al. (2023) (Zhang et al., 2023)

- **Problem:** KG embeddings can leak individual triples during downstream tasks.
- **Implementation:** Adds differential privacy by injecting noise into gradients and loss functions during embedding training.
- **Limitations:**
  - Focuses on embedding privacy, not structural privacy.
  - Not suitable for ASTs, call graphs, or file/module dependency graphs.
- **Motivation for PEKG:** Reinforces the need to sanitize KG structure before interacting with LLMs not just embeddings.

Synthesis and Gaps in Literature

- Existing work covers:
  - Structural anonymization
  - Temporal anonymization
  - Differential privacy for embeddings
- **BUT none of these works address:**
  - KGs built from **ASTs, call graphs, imports, and repository metadata**
  - Privacy-preserving transformations for **LLM-based code summarization**
  - The unique leakage in software engineering graphs:
    - Identifier leakage
    - Developer/provenance leakage
- **Gap → Directly motivates PEKG:**
  Need a sanitization layer that preserves semantic relationships but removes sensitive info before any KG → LLM interaction.

Problem Statement & Objectives

Problem Statement:

- Existing code summarization pipelines rely on raw source code or raw KGs that expose sensitive identifiers, metadata, and architectural patterns.

Why Raw KG Still Leaks Privacy:

- Even without code, KGs reveal API names, developer information, repository structure, and internal system topology.

Phase-1 Objectives:

- Build an end-to-end KG and summarization pipeline to identify where and how privacy leakage occurs.

Phase-2 Objectives:

- Design and integrate a Privacy-Enhanced KG (PEKG) that sanitizes identifiers, literals, and structural cues before LLM interaction.

Current Output vs Expected PEKG-based Summarization

```
(venv) PS C:\Users\jsrin\OneDrive\Desktop\PEKG\Code_Summarization> python .\kg_aware_summarizer.py
=== KG-Aware Code Summarizer ===
Enter the file path (e.g., location.filename.py): dags/src/anomaly_code_handler.py

Query: summarize "dags/src/anomaly_code_handler.py"

Summary (KG-aware):
This module defines the function
  dags.src.anomaly_code_handler.handle_anomalous_codes,
a key component in the early stages of the data-cleaning DAG. The function
identifies and standardises anomalous invoice or transaction codes prior to
downstream modelling. Within the KG, this node is connected via CALLS and
DEFINES edges to the central Airflow DAG, and via DEPENDS_ON edges to other
cleaning and feature-engineering tasks. This situates the function as part of
a coordinated transformation pipeline rather than an isolated utility.

Provenance metadata shows that the function was introduced through commit
f01c5123c9453c3f3972d88a874ee5cdc05c8f16 authored by "Ashkan Ghanavati" and
merged via pull request #22. The commit summary ("create dags in airflow.py up
to anomaly_codes_task") indicates that anomaly handling was designed as a
first-class step in the DAG's initial architecture. KG enrichment further
reveals that no subsequent refactor commits modified this file, suggesting
stability in its function. Such contextual information—developer intent, code
stability, position in the workflow—cannot be recovered from file content
alone but becomes explicit through KG-driven summarisation.
```

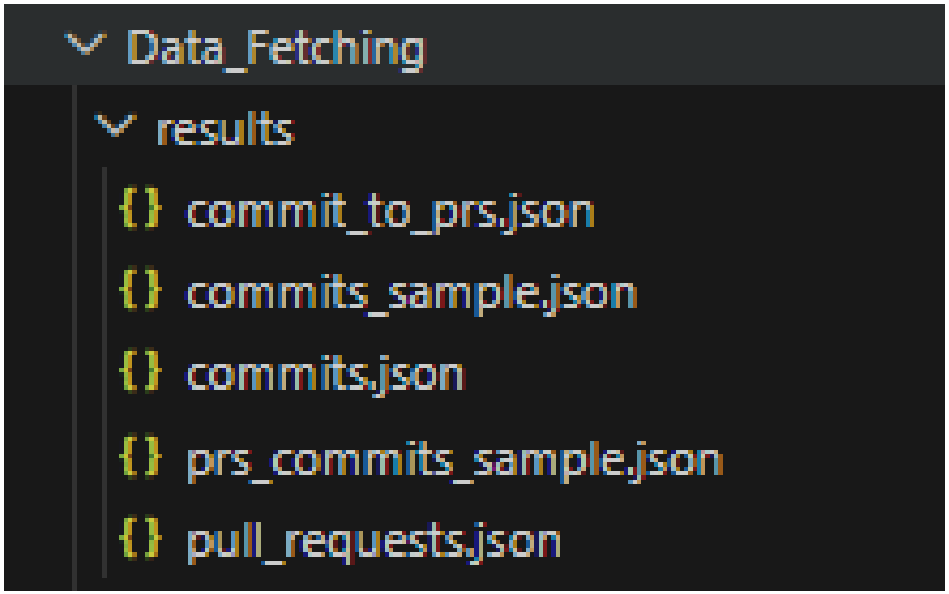
Expected Output After PEKG Sanitization

This module defines a function responsible for identifying and standardizing anomalous invoice or transaction codes as part of the early data-cleaning stages in a larger workflow. The function participates in a structured processing pipeline, interacting with downstream cleaning and feature-engineering components. Provenance metadata indicates that the function was added in an earlier version of the project during the initial construction of the workflow. Subsequent history suggests the module has remained stable, without major refactoring, indicating its consistent architectural role.

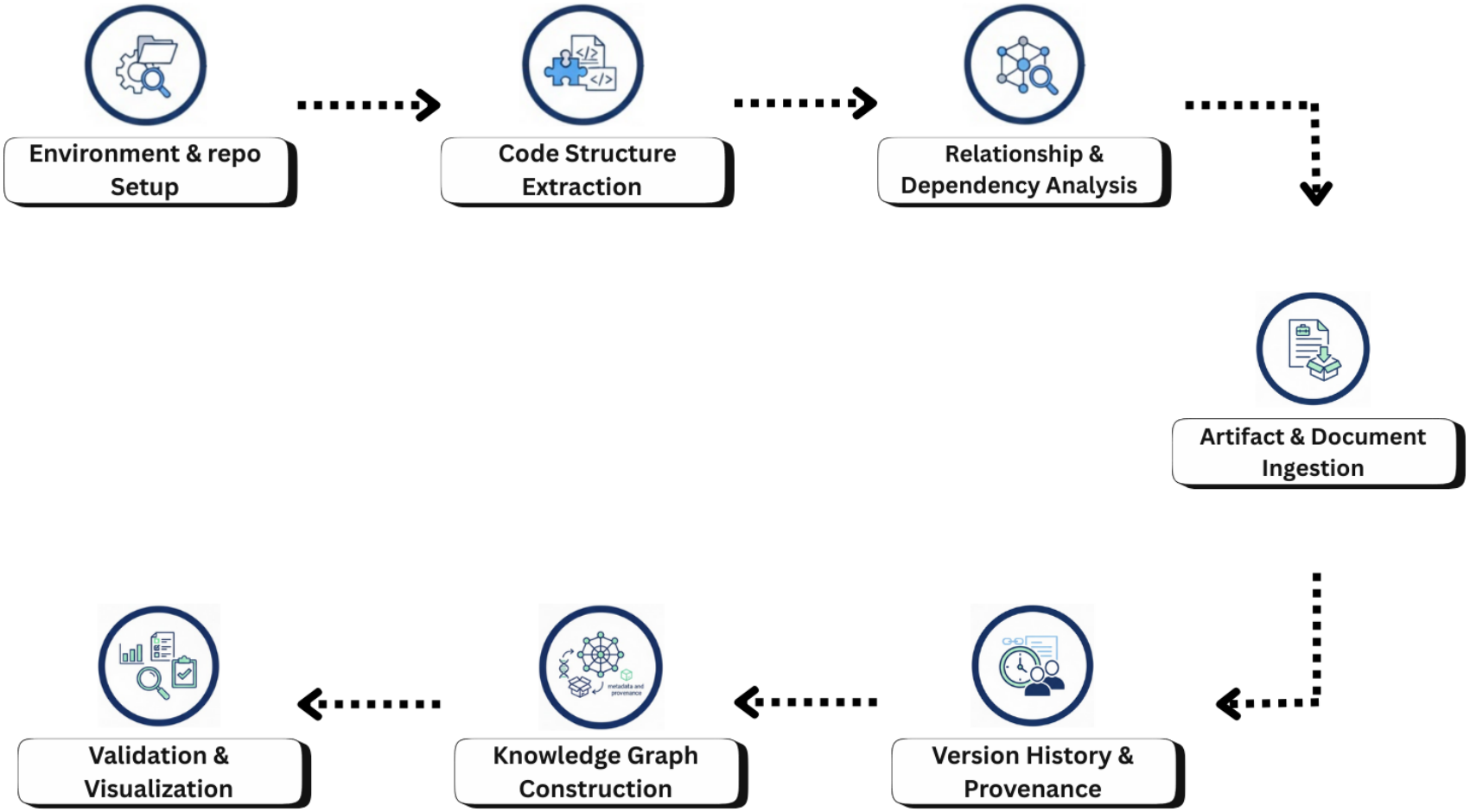
Methodology Overview

- **Repository Understanding:** Extract structure, metadata, and dependencies.
- **KG Construction:** Build nodes and edges from ASTs, calls, imports, and provenance.
- **Summarization Pipeline:** Resolve target $\rightarrow$ extract context $\rightarrow$ generate LLM summary.
- **Future PEKG Layer:** Sanitize identifiers, literals, and structure before LLM interaction.

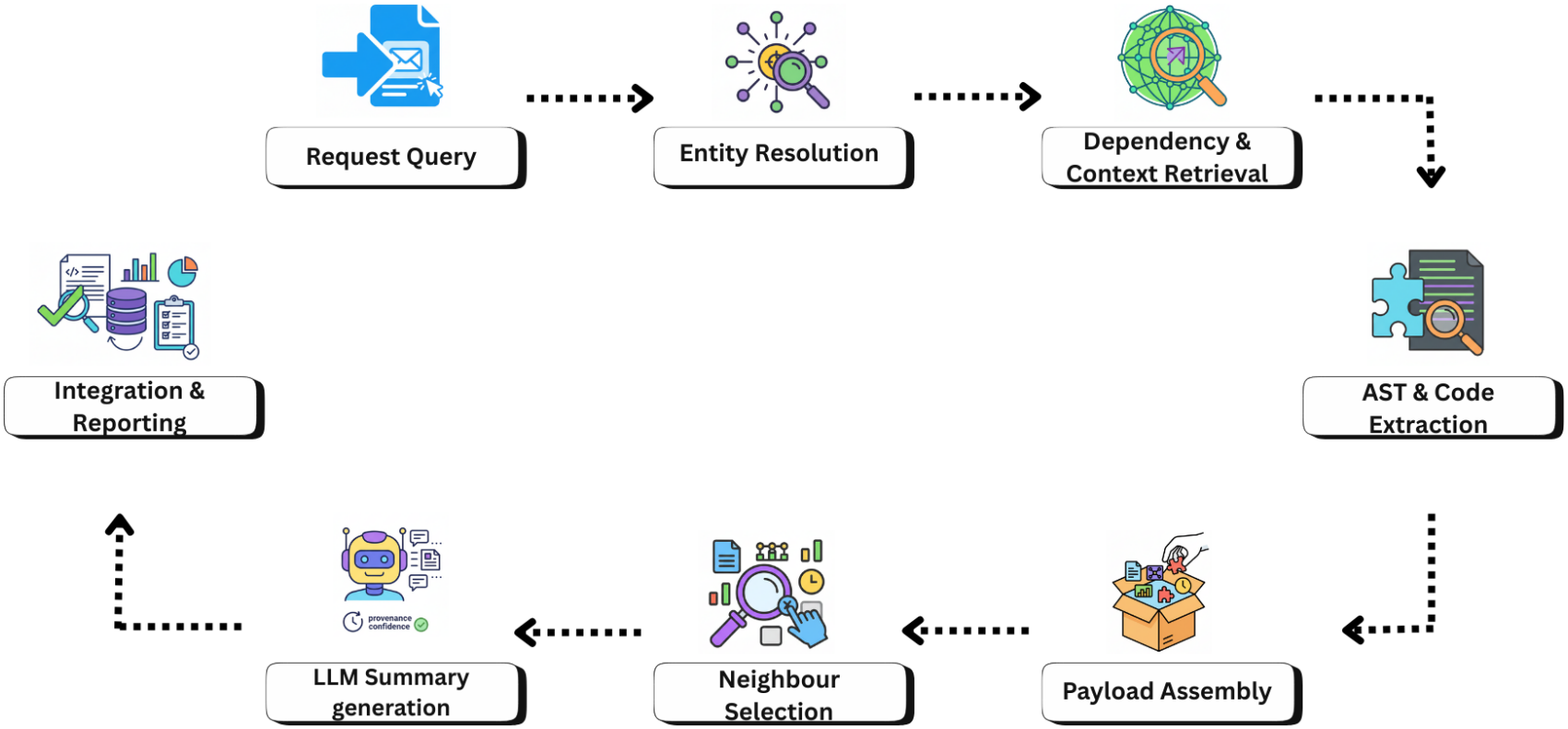
Phase-1: Repository & Metadata Extraction



Phase-1: Knowledge Graph Construction



Phase-1: Summarization Pipeline



KG Quality Evaluation: Node Distribution

Node Type	Count
Function	638
Commit	352
Document	305
File	250
Pull Request	109
Module	99
Package	54

Table 5.1: Node distribution in the constructed Knowledge Graph.

KG Quality Evaluation: Edge Distribution

Edge Type	Count
CLOSED_BY	1,280
DEFINES	548
MODIFIED_BY	311
CALLS	221
IMPORTS	87
DEPENDS_ON	48
MENTIONS	11

Table 5.2: Edge distribution in the constructed Knowledge Graph.

KG Quality Evaluation: JSON Representations

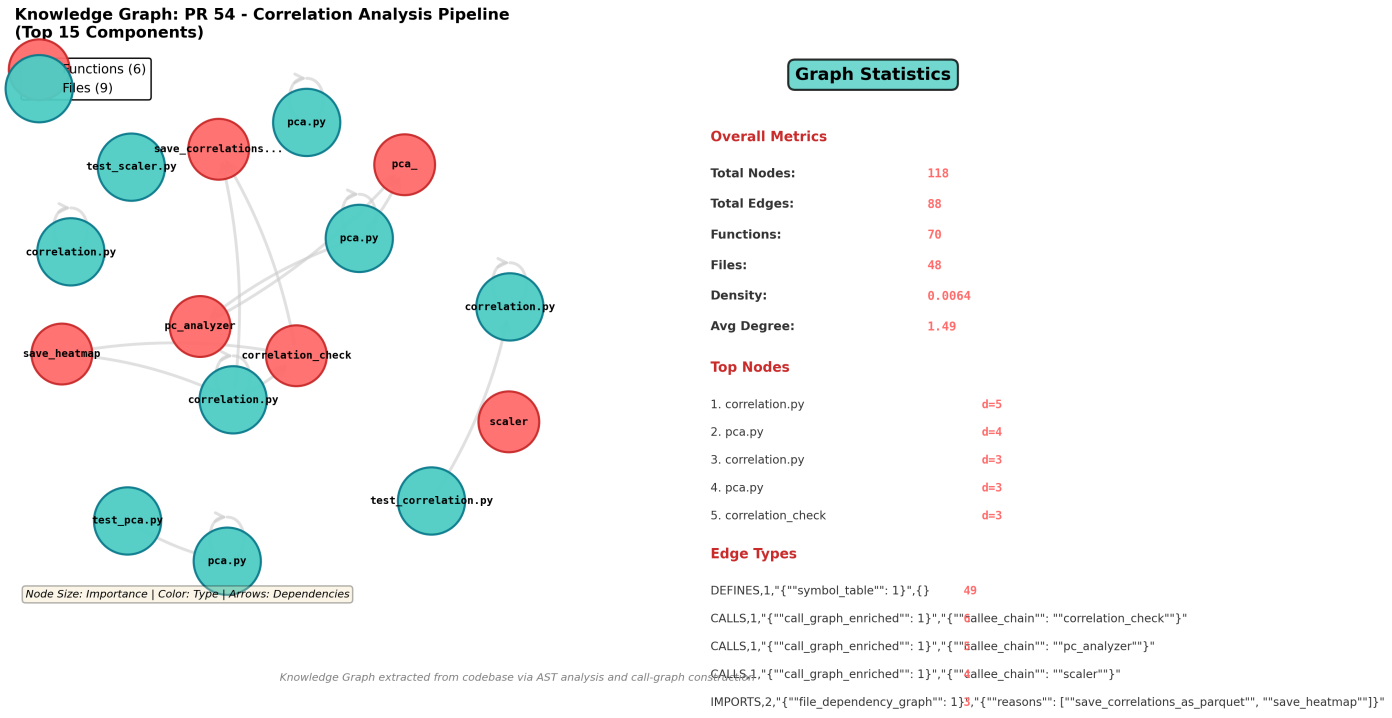
KG Node (Function)


```
{
  "node_id": "Function:dags.src.correlation.correlation_check",
  "type": "Function",
  "name": "correlation_check",
  "file": "dags/src/correlation.py",
  "module": "dags.src.correlation",
  "parameters": ["in_path", "out_path", "correlation_threshold"],
  "return_type": "None",
  "defined_in_commit": "0ce21a710a9b6cd46eb330cf6cd75dc24436faa3",
  "pr_number": 54
}
```


CALLS Edge


```
{
  "source": "Function:dags.src.correlation.correlation_check",
  "target": "Function:dags.src.correlation.save_heatmap",
  "type": "CALLS",
  "line_number": 42,
  "file": "dags/src/correlation.py"
}
```

KG Quality Evaluation: Subgraph Visualization



Dataset: GitHub link

Summarization Evaluation: Quantitative Metrics

Metric	Score
BLEU	21.54
ROUGE-1	36.20
ROUGE-2	10.11
ROUGE-L	26.03
chrF	45.54
METEOR	0.27

Table 5.5: Quantitative metrics for KG-aware code summarization.

KG-aware Summarization Example (1)

```
(venv) PS C:\Users\jsrin\OneDrive\Desktop\PEKG\Code_Summarization> python .\kg_aware_summarizer.py
=== KG-Aware Code Summarizer ===
Enter the file path (e.g., location.filenme.py): dags/src/anomaly_code_handler.py

Query: summarize "dags/src/anomaly_code_handler.py"

Summary (KG-aware):
This module defines the function
  dags.src.anomaly_code_handler.handle_anomalous_codes,
a key component in the early stages of the data-cleaning DAG. The function
identifies and standardises anomalous invoice or transaction codes prior to
downstream modelling. Within the KG, this node is connected via CALLS and
DEFINES edges to the central Airflow DAG, and via DEPENDS_ON edges to other
cleaning and feature-engineering tasks. This situates the function as part of
a coordinated transformation pipeline rather than an isolated utility.

Provenance metadata shows that the function was introduced through commit
f01c5123c9453c3f3972d88a874ee5cdc05c8f16 authored by "Ashkan Ghanavati" and
merged via pull request #22. The commit summary ("create dags in airflow.py up
to anomaly_codes_task") indicates that anomaly handling was designed as a
first-class step in the DAG's initial architecture. KG enrichment further
reveals that no subsequent refactor commits modified this file, suggesting
stability in its function. Such contextual information—developer intent, code
stability, position in the workflow—cannot be recovered from file content
alone but becomes explicit through KG-driven summarisation.
```

KG-aware Summarization Example (2)

```
(venv) PS C:\Users\jsrin\OneDrive\Desktop\PEKG\Code_Summarization> python .\kg_aware_summarizer.py
=== KG-Aware Code Summarizer ===
Enter the file path (e.g., location.filenme.py): dags/src/cancellation_details.py

Query: summarize "dags/src/cancellation_details.py"

Summary (KG-aware):
This module implements
  dags.src.cancellation_details.cancellation_details,
which enriches transaction records with cancellation-related attributes. The
KG positions this function within a cluster of behavioural analytics tasks,
with edges linking it to loaders, outlier handlers, and RFM metric generators.
This reveals that the cancellation logic forms part of a broader behavioural
feature-extraction pipeline, rather than being an isolated post-processing
step.

The provenance layer shows that the file was introduced by commit
06d80485b7a621184ec3d4e83a2836f2da6d3387 authored by "Moheth2000" and merged
via pull request #40. The associated commit summary ("The scripts have been
added to dags") reflects a bulk integration of multiple data-processing
components. Unlike the anomaly handler, this file shows multiple MODIFIED_BY
edges in the KG, indicating later refinements as the analytics pipeline
evolved. By combining content-level insights with socio-technical metadata,
the KG enables summarisation that captures not only what the code does, but
also its historical origin, evolution, and its role in the larger ecosystem.
```

Privacy Leakage Analysis

- **Identifier Leakage** – Exposes internal class names, function names, service identifiers, and API endpoints.
- **Literal Leakage** – Reveals URLs, access tokens, credentials, configuration values, and file paths.
- **Provenance Leakage** – Includes developer names, commit messages, PR metadata, timestamps, and workflow activity.
- **Structural Leakage** – Discloses architecture, dependency chains, module relationships, and operational workflows.
- **Why KG Alone is Not Enough:**
  - Raw KGs still encode sensitive identifiers and structural clues.
  - AST $\rightarrow$ KGs preserve literals, function signatures, and dependency semantics.
  - Provenance subgraphs leak organizational and contributor details.

Additional Privacy Observations

- **Cross-Graph Leakage:** Combining AST, call graph, and provenance graphs amplifies inference risks.
- **Transitive Exposure:** Even anonymized nodes can be re-identified through neighboring structures.
- **Pattern Leakage:** Common patterns (naming, folder layout, modular boundaries) reveal internal architecture.
- **Behavioral Leakage:** Function-call chains expose pipeline logic and proprietary workflows.
- **Repository Evolution Leakage:** Commits and PR graphs expose development cycles and operational cadence.

Contributions & Limitations (Phase-1)

Contributions

- Built an end-to-end pipeline for repo analysis, KG construction, and KG-aware summarization.
- Constructed a multi-layer KG (AST, CALLS, IMPORTS, commits, PRs).
- Performed quantitative (BLEU, ROUGE, METEOR, chrF) and qualitative evaluations.
- Identified privacy leakage patterns across identifiers, literals, provenance, and structure.

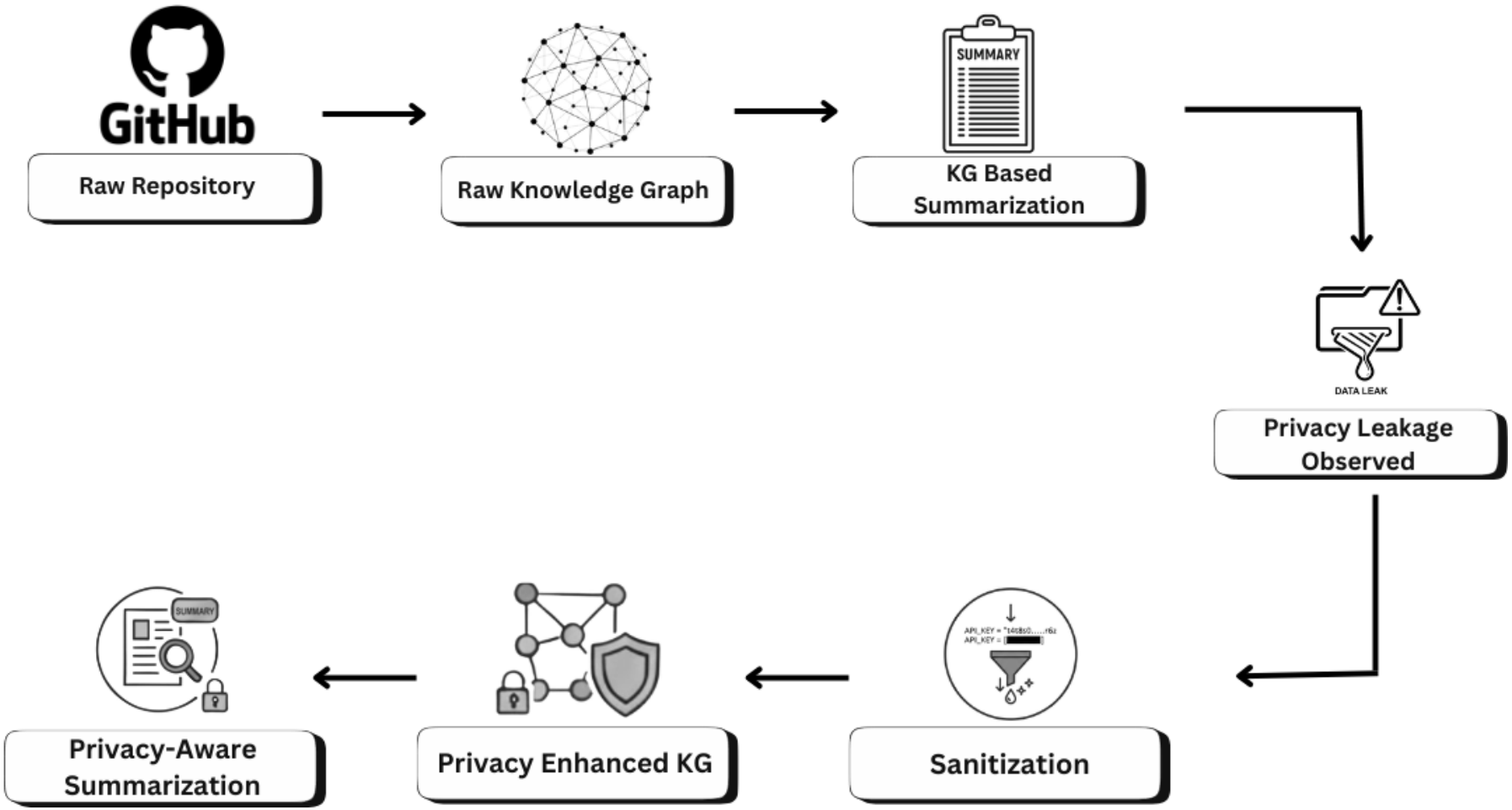
Limitations

- KG still contains raw identifiers and sensitive literals.
- LLM summaries may leak architectural or domain semantics.
- No privacy–utility tradeoff evaluation yet.
- Limited to Python repositories and AST-based extraction.

Future Work (Phase-2)

- Implement the Privacy-Enhanced Knowledge Graph (PEKG) sanitization layer.
- Add static sanitization for URLs, tokens, API keys, and sensitive literals.
- Design rarity-based identifier abstraction to mask domain-specific names.
- Evaluate privacy–utility tradeoffs for different sanitization levels.
- Extend KG pipeline to support multilingual repositories and temporal evolution.

Proposed PEKG Architecture (Phase-2)



Sanitization Strategy (PEKG)(Phase-2)

- **Stage 1: Static Sanitization**
  - Regex-based masking of URLs, tokens, secrets, email IDs, hostnames.
  - Applied across all files, AST literals, and KG node attributes.
- **Stage 2: Identifier Abstraction**
  - Compute rarity scores for function/class/module names.
  - Mask rare or domain-specific identifiers with neutral placeholders.
  - Preserve CALLS/IMPORTS/DEPENDS_ON relationships for utility.
- **Outcome:** Structure preserved, sensitive semantics removed.

Conclusion

- Phase-1 delivered a complete KG construction and KG-aware summarization pipeline.
- Evaluation revealed multiple privacy leakages across identifiers, literals, provenance, and structure.
- These findings strongly motivate the need for a Privacy-Enhanced KG (PEKG).
- Phase-2 will focus on implementing the sanitization strategy and evaluating privacy–utility tradeoffs.

References I

Bellomarini, L., Diligenti, M., Gentili, R., et al. (2024). Privacy-preserving synthetically augmented knowledge graphs. In *Proceedings of the VLDB Endowment*.

Zhang, J., Ma, Q., Wei, X., et al. (2023). A framework for differentially private knowledge graph embeddings. *Journal of Web Semantics*.

Thank You!